

DIPARTIMENTO DI INFORMATICA

Corso di Laurea in Informatica

Tesi di Laurea

Studio e verifica dei flussi IEC 61850 tramite libIEC61850 e analisi Wireshark

Relatore universitario:

Prof. Elvio Gilberto Amparore

Relatore aziendale:

Dott. Federico Cerruti

Sede del tirocinio:

Col Group – Trofarello

Candidato:

Mat. 867140

Plaia Antonino

Periodo di svolgimento: 28/09/2026 – 05/11/2026

ANNO ACCADEMICO 2025/2026

Indice

1	Introduzione	1
1.1	Automazione di sottostazione	1
1.2	Il ruolo degli IED	1
1.3	Motivazione della validazione dei flussi IEC 61850	2
1.4	La BU9020 in sintesi	2
1.5	Obiettivi del lavoro	2
2	IEC 61850 e comunicazione tra IED	4
2.1	Lo standard e le sue parti	4
2.2	Il modello dati IEC 61850	4
2.2.1	Logical Node	5
2.2.2	Data set	5
2.3	I servizi di comunicazione	5
2.4	MMS	5
2.5	GOOSE	5
2.6	Configurazione: i file SCL	6
2.7	Architettura della sottostazione	6
2.8	Validazione funzionale e validazione protocollare	7
3	Architettura del banco di prova	8
3.1	Panoramica	8
3.2	La BU9020	8
3.3	L'SBC ARM Linux	9
3.4	La piattaforma di simulazione dei segnali	9
3.5	Collegamento seriale	9
3.6	Rete Ethernet condivisa	9
3.7	Strumenti software usati	9
4	Implementazione del sistema di test	10
4.1	L'orchestratore	10
4.2	Protocollo seriale verso l'ESP32	10
4.3	Integrazione di libIEC61850	10
4.4	Cattura con Wireshark e tshark	10
4.5	Formato degli scenari	11

4.6	Gestione dei risultati	12
4.7	Interfaccia web (componente opzionale)	12
5	Validazione sperimentale	13
5.1	Scenari selezionati	13
5.2	Procedura di esecuzione	13
5.3	Risultati	13
5.4	Analisi dei frame GOOSE	14
5.5	Correlazione tra stimolo, evento e messaggio	14
5.6	Casi di errore o comportamento inatteso	14
6	Conclusioni	15
6.1	Valutazione del lavoro	15
6.2	Sviluppi futuri	15
6.3	Bilancio dell'esperienza formativa	15
	Bibliografia	16

Capitolo 1

Introduzione

1.1 Automazione di sottostazione

Una sottostazione elettrica è il nodo della rete di trasmissione e distribuzione in cui la tensione viene trasformata, l'energia smistata e i guasti isolati. Le apparecchiature che la compongono, come interruttori, sezionatori, trasformatori di misura e relè di protezione, sono controllate e monitorate da dispositivi elettronici che, collegati in rete, formano il sistema di automazione della stazione (*Substation Automation System*).

Storicamente lo scambio di informazioni tra questi dispositivi è avvenuto tramite protocolli diversi, spesso specifici del costruttore, e attraverso cablaggi in rame dedicati a ogni singolo segnale. L'integrazione di apparati di produttori differenti richiedeva quindi dispositivi di conversione di protocollo (*gateway*), che introducono ritardi e possibili errori nello scambio dei dati e aumentano i costi di progettazione, messa in servizio e manutenzione.

Per superare questi limiti la Commissione Elettrotecnica Internazionale (IEC) ha sviluppato lo standard IEC 61850, *Communication networks and systems for power utility automation* [1]. Il suo obiettivo è l'interoperabilità: dispositivi di costruttori diversi devono poter scambiare informazioni senza conversioni, indipendentemente dalle caratteristiche fisiche di ciascun apparato. Lo standard è descritto nel Capitolo 2.

1.2 Il ruolo degli IED

I dispositivi che svolgono le funzioni di automazione sono detti IED (*Intelligent Electronic Device*): apparati elettronici a microprocessore che realizzano funzioni di protezione, controllo, misura e monitoraggio sulle apparecchiature primarie e che dispongono di un'interfaccia di comunicazione. Sono IED, ad esempio, i relè di protezione, le unità di controllo di stallo e i regolatori di tensione.

Gli IED scambiano informazioni in due direzioni: tra loro, ad esempio per gli interblocchi e per i segnali di sgancio, e con il sistema di supervisione della stazione, a cui inviano misure, stati e allarmi e da cui ricevono i comandi. Poiché dal corretto scambio di queste informazioni dipende il funzionamento dell'impianto, la comunicazione di un IED è parte integrante della sua funzionalità e non un semplice accessorio.

1.3 Motivazione della validazione dei flussi IEC 61850

L'interoperabilità prevista dallo standard non è automatica. Ogni costruttore implementa un sottoinsieme dei servizi previsti e configura in modo proprio i dati esposti; un errore nella pubblicazione di un messaggio, come uno stato non coerente con la condizione reale, una ritrasmissione mancata o una numerazione sbagliata, può restare inosservato fino a quando un altro dispositivo non reagisce in modo scorretto. Questo vale in particolare per i messaggi GOOSE, che trasportano segnali critici come interblocchi e sgancio.

È quindi importante verificare che un dispositivo pubblichi i messaggi attesi, nel momento atteso e con il contenuto atteso. L'ispezione manuale del traffico con un analizzatore di rete è utile per la diagnosi, ma è lenta, dipende dall'operatore ed è difficile da ripetere a ogni modifica del firmware. Da qui l'esigenza di una procedura automatica e ripetibile, che metta in relazione lo stimolo applicato al dispositivo con il traffico che ne risulta.

1.4 La BU9020 in sintesi

Il dispositivo oggetto della verifica è la BU9020, un'unità destinata ad applicazioni di stallo in ambito elettrico. Fornisce funzioni di controllo e monitoraggio, tra cui il controllo di sincronismo, la richiusura automatica, la regolazione della tensione del trasformatore, la gestione di ingressi e uscite digitali e la pubblicazione e ricezione di messaggi IEC 61850. L'unità è descritta in dettaglio nel Capitolo 3. **[DA COMPLETARE: eventuale ruolo della BU9020 nell'offerta dell'azienda]**

1.5 Obiettivi del lavoro

Il presente lavoro nasce dal tirocinio curricolare svolto presso Col Group, a Trofarello, nel periodo dal 28 settembre al 5 novembre 2026. **[DA COMPLETARE: L'azienda è molto grande avendo molte sedi, e il loro lavoro si articola in molti ambiti. In particolare, avendo come clienti aziende come Enel e Terna, si occupano della produzione di soluzioni per le sottostazioni elettriche. Si occupano di creare nuove soluzioni agendo a livello elettronico, tramite un lavoro di affinamento, modifiche sull'hardware, e lo sviluppo delle componenti software di base, creano i cassetti elettrici che monitorano e agiscono sulla corrente dei grandi impianti elettrici. Il reparto con cui mi sono interfacciato è quello di ricerca e sviluppo, concentrato sulla parte di omologazione dei nuovi sistemi hardware e software.]**

L'obiettivo è progettare e realizzare un ambiente di test per gestire una scheda di segnali analogici e digitali e analizzare i flussi IEC 61850 generati o ricevuti dalla BU9020 durante scenari funzionali controllati. Il lavoro correla tre livelli:

1. **Stimolo fisico simulato.** Comandi inviati via seriale a una piattaforma basata su ESP32, che generano condizioni analogiche e digitali in ingresso all'unità.
2. **Comportamento applicativo della BU9020.** Attivazione di funzioni come sincronismo, richiusura o regolazione di tensione in risposta agli stimoli.

3. Traffico IEC 61850 sulla LAN. Messaggi MMS e GOOSE osservati tramite un client basato su libIEC61850 [2] e tramite cattura con Wireshark [3].

Il risultato atteso non è una semplice cattura passiva del traffico, ma una procedura ripetibile in grado di stabilire se un determinato scenario produce il flusso IEC 61850 atteso. Il simulatore di segnali è già stato realizzato dall'azienda e non fa parte del contributo del lavoro: questo consiste nello studio del protocollo e nella costruzione di un *framework* che automatizzi il ciclo di verifica.

Le attività svolte sono state **[DA COMPLETARE: riassumere in 3-5 righe cosa è stato fatto]**. I risultati principali sono **[DA COMPLETARE: da scrivere a lavoro concluso, con dati concreti]**.

Capitolo 2

IEC 61850 e comunicazione tra IED

Questo capitolo introduce i concetti dello standard necessari per comprendere il lavoro: il modello dati, i servizi MMS e GOOSE, il ruolo dei file di configurazione e la differenza tra validazione funzionale e protocollare. [DA COMPLETARE: edizione dello standard supportata dal dispositivo in esame]

2.1 Lo standard e le sue parti

Lo standard IEC 61850 si fonda su tre idee principali: un modello dati virtualizzato, la separazione tra servizi di comunicazione e protocolli concreti, un linguaggio formale di configurazione. È suddiviso in più parti; quelle rilevanti per il lavoro sono riassunte nella Tabella 2.1.

Parte	Contenuto
6	Linguaggio di configurazione SCL
7-2	Servizi astratti ACSI e modello informativo di base
7-3	Classi di dati comuni
7-4	Classi di nodi logici compatibili e oggetti dato
8-1	Associazione dei servizi a MMS ed Ethernet (client-server e GOOSE)
9-2	Associazione dei <i>Sampled Values</i> a Ethernet
10	Test di conformità

Tabella 2.1: Parti dello standard IEC 61850 rilevanti per il lavoro svolto.

2.2 Il modello dati IEC 61850

Ogni dispositivo viene descritto in modo astratto, in base alle funzioni che svolge e alle informazioni che espone, e non in base al suo hardware. Questa virtualizzazione è ciò che permette a dispositivi diversi di presentarsi allo stesso modo a chi li interroga.

Un dispositivo fisico (IED) contiene uno o più *Logical Device*, ciascuno composto da *Logical Node*. I nodi logici contengono *Data Object*, a cui sono associati *Data Attribute*, con nomi e significati uniformi per tutti i costruttori. Un riferimento a un attributo ha quindi la forma `LogicalDevice/LogicalNode.DataObject.DataAttribute`.

2.2.1 Logical Node

Un nodo logico rappresenta una singola funzione dell'apparato. Ad esempio XCBR modella un interruttore, MMXU le misure, RSYN il controllo di sincronismo, RREC la richiusura automatica e ATCC la regolazione automatica del variatore di prese del trasformatore. Questi ultimi nodi corrispondono alle funzioni della BU9020 oggetto del lavoro. **[DA COMPLETARE: verificare nel file ICD quali nodi logici espone effettivamente l'unità in esame]**

2.2.2 Data set

I dati che devono essere trasmessi insieme vengono raggruppati in un *data set*, ossia un elenco ordinato di riferimenti a dati o ad attributi. Il data set è l'unità di informazione pubblicata dai messaggi GOOSE e dai rapporti MMS: il suo contenuto e il suo ordine determinano ciò che il ricevitore può leggere.

2.3 I servizi di comunicazione

I servizi di scambio informativo sono definiti in forma astratta (ACSI, *Abstract Communication Service Interface*) e poi associati a protocolli concreti. I principali sono:

- MMS (*Manufacturing Message Specification*) su TCP/IP, per la comunicazione client-server;
- GOOSE (*Generic Object Oriented Substation Event*), per messaggi rapidi in multicast direttamente su Ethernet, usati per interblocchi e segnali di sgancio tra protezioni;
- *Sampled Values* (SV), per la trasmissione dei campioni digitalizzati di correnti e tensioni.

Il presente lavoro riguarda MMS e GOOSE, descritti nelle sezioni seguenti.

2.4 MMS

Il servizio è basato sul modello client-server ed è trasportato su TCP/IP (tipicamente sulla porta 102). Un client, ad esempio un sistema di supervisione, può leggere e scrivere i valori dei dati dell'IED, inviare comandi e ricevere rapporti (*report*) spontanei. I rapporti sono configurati tramite *Report Control Block*, bufferizzati o non bufferizzati, che inviano i dati al client al verificarsi di determinate condizioni, come la variazione di un valore.

2.5 GOOSE

Il servizio segue il modello *publisher-subscriber*: un IED pubblica il contenuto di un data set e gli altri IED interessati lo ricevono. I messaggi viaggiano direttamente su Ethernet (livello 2), senza passare da IP e TCP, in multicast e con un EtherType dedicato, per ridurre la latenza. In caso di cambio di stato il messaggio viene inviato subito e poi ritrasmesso a intervalli inizialmente brevi e progressivamente crescenti, fino a un intervallo massimo di regime, così da recuperare eventuali perdite senza attendere conferme.

I campi del messaggio rilevanti per la verifica sono:

- **gocbRef**: riferimento al *GOOSE Control Block* che pubblica il messaggio;
- **datSet**: riferimento al data set pubblicato;
- **goID**: identificatore del messaggio GOOSE;
- **timeAllowedToLive**: tempo massimo che il ricevitore attende prima di considerare perso il messaggio successivo;
- **t**: marca temporale dell'ultimo cambio di stato;
- **stNum**: numero di stato, incrementato a ogni cambio di valore del data set;
- **sqNum**: numero di sequenza, incrementato a ogni ritrasmissione con lo stesso **stNum** e riportato a zero quando **stNum** cambia;
- il contenuto del data set (*payload* applicativo).

2.6 Configurazione: i file SCL

Il linguaggio SCL (*Substation Configuration Language*), basato su XML, permette di descrivere in modo formale i dispositivi, la sottostazione e la rete di comunicazione, e di scambiare le configurazioni tra strumenti di costruttori diversi. I file di configurazione hanno estensioni diverse a seconda dello scopo:

- ICD (*IED Capability Description*): le capacità di un singolo IED, fornito dal costruttore;
- CID (*Configured IED Description*): la configurazione di un IED specifico;
- SCD (*Substation Configuration Description*): la descrizione dell'intero sistema, con i dispositivi e i collegamenti tra i loro dati.

Per chi sviluppa un sistema di test questi file sono la fonte da cui ricavare i nomi dei nodi logici, dei data set e dei *Control Block* da verificare. **[DA COMPLETARE: il file .icd contiene la configurazione di base dell' IED ed è il punto di partenza dal quale si sviluppa la prima analisi. Tramite un parser raccogliamo le informazioni di base e creiamo le configurazioni di base che verranno popolate con i dati MMS e Goose subito dopo la loro creazione.]**

2.7 Architettura della sottostazione

L'architettura di una sottostazione IEC 61850 è organizzata su tre livelli: livello di stazione, livello di campo (*bay*, in italiano stallo) e livello di processo, collegati da reti Ethernet dette bus di stazione e bus di processo. Le funzioni di controllo di stallo, come quelle dell'unità considerata in questa tesi, si collocano al livello di campo.

2.8 Validazione funzionale e validazione protocollare

Verificare un dispositivo IEC 61850 può significare due cose diverse.

La **validazione protocollare** controlla che i messaggi scambiati rispettino le regole del protocollo: struttura dei frame, presenza e valori ammissibili dei campi, coerenza della numerazione (`stNum`, `sqNum`), comportamento delle ritrasmissioni. La parte 10 dello standard definisce le procedure di test di conformità di questo tipo.

La **validazione funzionale** controlla che il dispositivo, in seguito a una condizione di ingresso, produca l'effetto atteso: lo stato corretto, pubblicato al momento corretto. Un dispositivo può infatti emettere frame formalmente corretti con un valore sbagliato rispetto alla situazione reale.

Il banco di prova realizzato opera su entrambi i piani: verifica la struttura dei frame GOOSE e la loro coerenza con lo stimolo applicato. Non è tuttavia una procedura di certificazione secondo la parte 10, ma una verifica funzionale in scenari controllati. **[DA COMPLETARE: confermare questa impostazione con il relatore aziendale]**

Capitolo 3

Architettura del banco di prova

3.1 Panoramica

L'architettura, illustrata in Figura 3.1, prevede un computer a scheda singola (SBC) ARM con sistema operativo Linux che svolge il ruolo di orchestratore. L'SBC è collegato in due modi: via seriale (USB/UART) alla piattaforma ESP32, che genera i segnali analogici e digitali applicati alla BU9020, e via Ethernet alla stessa LAN della BU9020, per interrogarla con il client IEC 61850 e catturare il traffico.

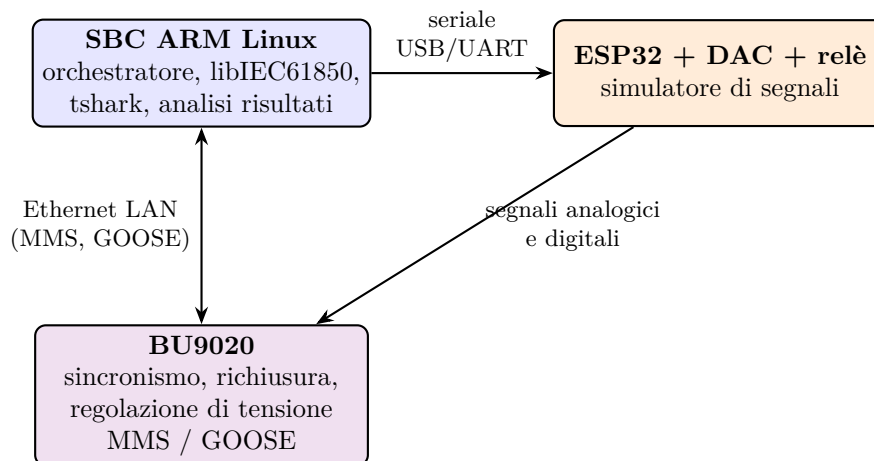


Figura 3.1: Architettura del banco di prova.

3.2 La BU9020

La BU9020 è l'unità sotto verifica. È destinata ad applicazioni di stallo in ambito elettrico e fornisce funzioni di controllo e monitoraggio: controllo di sincronismo, richiusura automatica, regolazione della tensione del trasformatore, gestione di ingressi e uscite digitali, pubblicazione e ricezione di messaggi IEC 61850. **[DA COMPLETARE: descrizione più completa: interfacce, versione del firmware, nodi logici e data set esposti, GOOSE Control Block configurati]**

[DA COMPLETARE: descrivere il cassetto elettrico che ospita la BU9020, le connessioni fisiche con la piattaforma di simulazione e la modalità di accesso al sistema operativo del cassetto]

3.3 L'SBC ARM Linux

L'SBC è l'orchestratore del banco: esegue il software di gestione dei test, comunica con il simulatore, interroga la BU9020 e cattura il traffico di rete. **[DA COMPLETARE: modello della scheda, sistema operativo, interfacce di rete e porte seriali utilizzate]**

3.4 La piattaforma di simulazione dei segnali

Per stimolare la BU9020 l'azienda ha già realizzato una piattaforma di prova basata su un microcontrollore ESP32, un convertitore digitale-analogico (DAC) e dei relè. La piattaforma è in grado di generare segnali analogici trifase e stati digitali controllati e può essere pilotata tramite interfaccia UART. **[DA COMPLETARE: grandezze generabili e relativi limiti (ad esempio la frequenza), numero di ingressi digitali]**

3.5 Collegamento seriale

L'orchestratore comanda il simulatore via seriale (USB/UART): è l'unico canale con cui lo stimolo fisico viene definito. **[DA COMPLETARE: parametri della connessione e formato dei comandi, descritti nella Sezione 4.2]**

3.6 Rete Ethernet condivisa

L'SBC e la BU9020 si trovano sulla stessa rete Ethernet. Poiché i messaggi GOOSE sono multicast di livello 2, restano confinati al dominio di broadcast, o alla VLAN, in cui vengono pubblicati: l'SBC deve quindi appartenere allo stesso segmento per poterli ricevere e catturare. **[DA COMPLETARE: topologia effettiva: switch, VLAN, indirizzi IP, eventuale uso di mirroring di porta]**

3.7 Strumenti software usati

Il lavoro si basa su tre strumenti principali:

- **libIEC61850** [2]: libreria open source scritta in C che implementa i servizi MMS, GOOSE e SV, usata per realizzare il client;
- **Wireshark e tshark** [3]: analizzatore di rete e relativo strumento a riga di comando, usati per la cattura e per l'estrazione automatica dei campi dai frame;
- **Python**: linguaggio dell'orchestratore e della suite di test. **[DA COMPLETARE: confermare la scelta del linguaggio e dell'eventuale framework di test, ad esempio Robot Framework]**

[DA COMPLETARE: versioni dei software utilizzati]

Capitolo 4

Implementazione del sistema di test

4.1 L'orchestratore

L'orchestratore è un programma, o un insieme di script, preferibilmente in Python. Il suo ciclo di lavoro prevede di: caricare la descrizione di uno scenario di test; inviare i comandi seriali alla piattaforma ESP32; avviare la cattura del traffico di rete; interrogare la BU9020 tramite il client IEC 61850; raccogliere risultati e registri (*log*); generare un rapporto di esito. **[DA COMPLETARE: struttura del software, moduli, scelte progettuali]**

4.2 Protocollo seriale verso l'ESP32

[DA COMPLETARE: formato dei comandi, risposte del simulatore, gestione di errori e temporizzazioni]

4.3 Integrazione di libIEC61850

Il client, basato su libIEC61850, deve consentire almeno la connessione MMS alla BU9020, la lettura di oggetti dato, la verifica di attributi IEC 61850 e la correlazione dei valori letti con gli eventi generati. Sono inoltre previsti, se supportati dal setup, l'attivazione o il controllo di un *Report Control Block* e la sottoscrizione o l'analisi dei messaggi GOOSE.

Un esempio di verifica è il seguente. All'evento simulato «perdita di sincronismo tra tensione di linea e di sbarra» deve corrispondere l'aggiornamento dello stato logico interno, l'eventuale blocco del comando, la pubblicazione di un messaggio GOOSE con stato «non sincronizzato» e la lettura via MMS di valori coerenti. La parte più interessante del lavoro è trasformare il client da semplice strumento di lettura a componente di verifica automatica. **[DA COMPLETARE: funzionalità effettivamente implementate e come il client è integrato nell'orchestratore]**

4.4 Cattura con Wireshark e tshark

Wireshark rimane lo strumento di riferimento per la validazione manuale e diagnostica, mentre **tshark** permette di estrarre automaticamente i campi dai file di cattura **.pcapng**. Un esempio di comando è il seguente:

```
tshark -r test_sync_check_ok.pcapng \
-Y "goose" \
-T fields \
-e frame.time_epoch \
-e eth.src \
-e goose.gocbRef \
-e goose.datSet \
-e goose.goID \
-e goose.stNum \
-e goose.sqNum
```

Le verifiche previste sui frame catturati riguardano:

- la presenza dei frame GOOSE attesi;
- la correttezza dei campi `gocbRef`, `datSet`, `goID`, `stNum`, `sqNum`, `timeAllowedToLive`, della marca temporale e del contenuto applicativo;
- la sequenza dei messaggi dopo un evento e le ritrasmissioni dopo un cambio di stato;
- la coerenza tra cambio di stato e incremento di `stNum`;
- la latenza tra l'evento e il frame GOOSE corrispondente;
- l'assenza di frame inattesi.

[DA COMPLETARE: filtri e comandi effettivamente usati, come le verifiche sono automatizzate]

4.5 Formato degli scenari

Gli scenari di test sono descritti in un file YAML o JSON che contiene gli stimoli da applicare, le condizioni attese e i criteri di superamento o fallimento. La struttura seguente è puramente indicativa e va sostituita con il formato effettivamente adottato.

```
scenario: <nome dello scenario>
stimoli:
  - comando: <comando seriale verso l'ESP32>
    attesa_ms: <ritardo prima del passo successivo>
attese:
  mms:
    - riferimento: <percorso dell'attributo>
      valore: <valore atteso>
  goose:
    - gocbRef: <riferimento del control block>
      incremento_stNum: <incremento atteso>
      latenza_max_ms: <soglia di latenza>
```

[DA COMPLETARE: formato reale degli scenari e scelte di progetto]

4.6 Gestione dei risultati

Ogni esecuzione deve essere tracciabile: l'esito, il file di cattura, il registro strutturato e la marca temporale vengono salvati insieme, così che il test sia riproducibile e i dati siano consultabili a posteriori. L'esito è riportato in un rapporto di superamento o fallimento (*pass/fail*) con i tempi misurati, i messaggi osservati e le eventuali anomalie. **[DA COMPLETARE: formato del rapporto e organizzazione dei file]**

4.7 Interfaccia web (componente opzionale)

Come obiettivo secondario, e solo se il tempo lo consente, l'orchestratore può esporre un'interfaccia web HTTP/REST in esecuzione sull'SBC. Il server web non comunica direttamente con l'ESP32: ogni comando passa attraverso l'orchestratore, che lo inoltra via seriale al simulatore. Le funzioni previste sono il controllo remoto del simulatore, un pannello di monitoraggio dello stato dei segnali e della connessione con la BU9020, la gestione degli scenari di test e un'API per l'automazione, che permette anche l'integrazione con sistemi di integrazione continua. **[DA COMPLETARE: indicare se questa componente è stata realizzata]**

Capitolo 5

Validazione sperimentale

5.1 Scenari selezionati

Sono previsti tre scenari, corrispondenti alle funzioni della BU9020 riassunte nella Tabella 5.1.

Scenario	Grandezze controllate	Verifiche IEC 61850
Controllo di sincronismo	differenza di fase e di tensione; differenza di frequenza, se supportata dalla piattaforma; stato digitale di abilitazione del comando	stato del consenso al sincronismo; eventuale GOOSE; lettura MMS dello stato associato; latenza tra stimolo e aggiornamento
Richiusura automatica	apertura dell'interruttore; simulazione di guasto; rientro delle condizioni corrette; abilitazione o inibizione della richiusura	sequenza degli stati; comandi e feedback associati; messaggi GOOSE generati; corretto incremento di <code>stNum</code> ; tempi tra gli eventi
Regolazione di tensione del trasformatore	tensione misurata; soglie di regolazione; ingressi digitali di blocco o consenso; eventuali feedback di posizione	stato del regolatore; richieste di salita e discesa prese; messaggi GOOSE relativi; valori MMS esposti; coerenza tra scenario e comando atteso

Tabella 5.1: Scenari di test previsti.

[DA COMPLETARE: indicare quali scenari sono stati effettivamente implementati e testati]

5.2 Procedura di esecuzione

[DA COMPLETARE: descrivere i passi di esecuzione di uno scenario, dalla configurazione iniziale al rapporto finale, e i criteri di pass/fail adottati]

5.3 Risultati

[**DA COMPLETARE:** per ogni scenario: esito, messaggi osservati, confronto con il comportamento atteso. Si può usare una tabella riassuntiva con scenario, esito, latenza misurata e note]

5.4 Analisi dei frame GOOSE

[**DA COMPLETARE:** analisi dei frame catturati: valori dei campi, andamento di `stNum` e `sqNum`, ritrasmissioni, eventuali figure di Wireshark annotate]

5.5 Correlazione tra stimolo, evento e messaggio

[**DA COMPLETARE:** come lo stimolo applicato, l'evento nella BU9020 e il messaggio osservato sono messi in relazione nel tempo, con il metodo di misura delle latenze e la sua incertezza]

5.6 Casi di errore o comportamento inatteso

[**DA COMPLETARE:** comportamenti inattesi della BU9020 o del banco di prova e loro analisi]

Capitolo 6

Conclusioni

6.1 Valutazione del lavoro

[DA COMPLETARE: efficacia dell'ambiente di test e limiti del lavoro]

6.2 Sviluppi futuri

[DA COMPLETARE: estensioni possibili, ad esempio l'integrazione in una pipeline di test più ampia]

6.3 Bilancio dell'esperienza formativa

[DA COMPLETARE: competenze tecniche e trasversali acquisite, difficoltà incontrate e come sono state risolte, collegamento con gli insegnamenti del corso di laurea]

Bibliografia

- [1] IEC, *IEC 61850: Communication networks and systems for power utility automation*, serie di norme. **[DA COMPLETARE: indicare le parti e le edizioni consultate]**
- [2] MZ Automation GmbH, *libIEC61850*, libreria open source. <https://libiec61850.com>. **[DA COMPLETARE: data di consultazione e versione]**
- [3] Wireshark Foundation, *Wireshark*. <https://www.wireshark.org>. **[DA COMPLETARE: data di consultazione e versione]**
- [4] CESI, *FIA Informatica e Automazione*, 2005. **[DA COMPLETARE: completare i dati del documento e citarlo nel testo dove è stato usato]**